

YouTube Virality Modeling — 4 Model Comparison

This notebook runs four class-aligned models on the API-enriched YouTube dataset:

- Elastic Net
- KNN Regressor
- Random Forest Regressor
- MLP Regressor

Primary target: `log1p(view_count)` using `yt_view_count` when available, otherwise `views`.

Note: To avoid leakage, this notebook **does not** use likes/comments as predictors when predicting views.

```
In [ ]: import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from sklearn.linear_model import ElasticNet
from sklearn.neighbors import KNeighborsRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.neural_network import MLPRegressor

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

pd.set_option('display.max_columns', 200)
pd.set_option('display.width', 200)
```

```
In [ ]: # ---- Load data ----
DATA_PATH = '../youtube_data_enriched.csv'
df = pd.read_csv(DATA_PATH)
print(f'Shape: {df.shape}')
print('Columns sample:', df.columns[:20].tolist())
```

```
In [ ]: # ---- Feature engineering + variable audit ----
def pick_col(primary, fallback, frame):
    return primary if primary in frame.columns else fallback
```

```

views_col = pick_col('yt_view_count', 'views', df)
title_col = pick_col('yt_title', 'title', df)
desc_col = pick_col('yt_description', 'description', df)
duration_col = pick_col('yt_duration_sec', 'duration', df)

df_model = df.copy()

# Safe datetime features
if 'yt_published_at' in df_model.columns:
    df_model['yt_published_at_dt'] = pd.to_datetime(df_model['yt_published_at'])
else:
    df_model['yt_published_at_dt'] = pd.NaT

if 'yt_channel_published_at' in df_model.columns:
    df_model['yt_channel_published_at_dt'] = pd.to_datetime(df_model['yt_channel_published_at'])
else:
    df_model['yt_channel_published_at_dt'] = pd.NaT

ref_time = df_model['yt_published_at_dt'].max()
if pd.isna(ref_time):
    ref_time = pd.Timestamp.utcnow()

df_model['video_age_days'] = (ref_time - df_model['yt_published_at_dt']).dt.days
df_model['channel_age_days'] = (ref_time - df_model['yt_channel_published_at_dt']).dt.days

# Text-length proxies
df_model['title_len'] = df_model[title_col].fillna('').astype(str).str.len()
df_model['desc_len'] = df_model[desc_col].fillna('').astype(str).str.len()

if 'hashtags' in df_model.columns:
    df_model['hashtag_count'] = (
        df_model['hashtags'].fillna('').astype(str).apply(lambda s: len(s.split('#')))
    )
else:
    df_model['hashtag_count'] = np.nan

# Candidate predictors (avoid target leakage from likes/comments)
candidate_features = [
    duration_col, 'bitrate', 'height', 'width', 'frame_rate',
    'yt_subscriber_count', 'yt_channel_view_count', 'yt_channel_video_count',
    'video_age_days', 'channel_age_days',
    'thumb_mean_brightness', 'thumb_colorfulness',
    'title_len', 'desc_len', 'hashtag_count',
    'category', 'codec', 'yt_channel_country',
    'yt_default_language', 'yt_default_audio_language',
    'yt_made_for_kids', 'yt_live_broadcast_content'
]

available_features = [c for c in candidate_features if c in df_model.columns]
missing_features = [c for c in candidate_features if c not in df_model.columns]

```

```

print(f'Target column: {views_col}')
print(f'Available features: {len(available_features)}')
print('Missing (if any):', missing_features)

# Build modeling frame
model_cols = available_features + [views_col]
model_df = df_model[model_cols].copy()

# Force numeric on known numeric columns if present
maybe_numeric = [
    duration_col, 'bitrate', 'height', 'width', 'frame rate',
    'yt_subscriber_count', 'yt_channel_view_count', 'yt_channel_video_
    'video_age_days', 'channel_age_days',
    'thumb_mean_brightness', 'thumb_colorfulness',
    'title_len', 'desc_len', 'hashtag_count', views_col
]
for c in maybe_numeric:
    if c in model_df.columns:
        model_df[c] = pd.to_numeric(model_df[c], errors='coerce')

# Drop rows with missing target
model_df = model_df.dropna(subset=[views_col]).copy()
model_df['target_log_views'] = np.log1p(model_df[views_col].clip(lower

print(f'Modeling rows after target filter: {len(model_df):,}')

```

```

In [ ]: # ---- Train/test split ----
X = model_df[available_features].copy()
y = model_df['target_log_views'].copy()

numeric_features = [c for c in X.columns if pd.api.types.is_numeric_dtype(X[c])]
categorical_features = [c for c in X.columns if c not in numeric_features]

print('Numeric features:', numeric_features)
print('Categorical features:', categorical_features)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print('Train shape:', X_train.shape, 'Test shape:', X_test.shape)

```

```

In [ ]: # ---- Preprocessors ----
scaled_numeric_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

unscaled_numeric_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median'))
])

```

```

cat_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

preprocessor_scaled = ColumnTransformer([
    ('num', scaled_numeric_pipe, numeric_features),
    ('cat', cat_pipe, categorical_features)
])

preprocessor_unscaled = ColumnTransformer([
    ('num', unscaled_numeric_pipe, numeric_features),
    ('cat', cat_pipe, categorical_features)
])

```

```

In [ ]: # ---- Four models (class-aligned) ----
model_specs = {
    'ElasticNet': {
        'preprocessor': preprocessor_scaled,
        'model': ElasticNet(alpha=0.01, l1_ratio=0.5, random_state=42,
    },
    'KNN': {
        'preprocessor': preprocessor_scaled,
        'model': KNeighborsRegressor(n_neighbors=25, weights='distance
    },
    'RandomForest': {
        'preprocessor': preprocessor_unscaled,
        'model': RandomForestRegressor(
            n_estimators=300,
            max_depth=20,
            min_samples_leaf=2,
            random_state=42,
            n_jobs=-1
        )
    },
    'MLP': {
        'preprocessor': preprocessor_scaled,
        'model': MLPRegressor(
            hidden_layer_sizes=(128, 64),
            activation='relu',
            learning_rate_init=0.001,
            max_iter=300,
            random_state=42
        )
    }
}

results = []
fitted_pipelines = {}

for name, spec in model_specs.items():
    pipe = Pipeline([

```

```

        ('prep', spec['preprocessor']),
        ('model', spec['model'])
    ])

    pipe.fit(X_train, y_train)
    preds = pipe.predict(X_test)

    rmse = mean_squared_error(y_test, preds, squared=False)
    mae = mean_absolute_error(y_test, preds)
    r2 = r2_score(y_test, preds)

    results.append({
        'model': name,
        'rmse_log_views': rmse,
        'mae_log_views': mae,
        'r2_log_views': r2
    })

    fitted_pipelines[name] = pipe
    print(f'{name} complete.')

results_df = pd.DataFrame(results).sort_values('rmse_log_views').reset_index()
results_df

```

Required Four Models Checklist

This project requires four core models. The checklist below confirms they are implemented in this notebook:

- Elastic Net
- KNN Regressor
- Random Forest Regressor
- MLP Regressor

```

In [ ]: # ---- Validate + mark required 4 models ----
required_models = ['ElasticNet', 'KNN', 'RandomForest', 'MLP']
implemented_models = list(model_specs.keys())

missing_required = [m for m in required_models if m not in implemented_models]

print('Required models implemented:')
for model_name in required_models:
    marker = '✅' if model_name in implemented_models else '❌'
    print(f' {marker} {model_name}')

assert not missing_required, f'Missing required models: {missing_required}'

# Add a core-model marker into results table for clear reporting
results_df['is_required_core_model'] = results_df['model'].isin(required_models)

```

results_df

Random Forest Tuning (GridSearchCV)

This step tunes Random Forest hyperparameters using 5-fold CV on the training set only.

You should keep this section for your final report so your RF result is defensible and not just a default configuration.

```
In [ ]: # ---- Grid search for Random Forest ----
rf_base_pipe = Pipeline([
    ('prep', preprocessor_unscaled),
    ('model', RandomForestRegressor(random_state=42, n_jobs=-1))
])

param_grid = {
    'model__n_estimators': [200, 400],
    'model__max_depth': [None, 20, 30],
    'model__min_samples_split': [2, 5],
    'model__min_samples_leaf': [1, 2],
    'model__max_features': ['sqrt', 0.6]
}

rf_grid = GridSearchCV(
    estimator=rf_base_pipe,
    param_grid=param_grid,
    cv=5,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1,
    verbose=1
)

rf_grid.fit(X_train, y_train)

print('Best RF params:', rf_grid.best_params_)
print('Best CV RMSE (log views):', -rf_grid.best_score_)

rf_best_pipe = rf_grid.best_estimator_
rf_best_preds = rf_best_pipe.predict(X_test)

rf_best_rmse = mean_squared_error(y_test, rf_best_preds, squared=False)
rf_best_mae = mean_absolute_error(y_test, rf_best_preds)
rf_best_r2 = r2_score(y_test, rf_best_preds)

print({'rmse_log_views': rf_best_rmse, 'mae_log_views': rf_best_mae, '
# Add tuned RF as another row for side-by-side comparison
results_df = pd.concat([
    results_df,
```

```

pd.DataFrame([
    {
        'model': 'RandomForest_GridSearch',
        'rmse_log_views': rf_best_rmse,
        'mae_log_views': rf_best_mae,
        'r2_log_views': rf_best_r2
    }
])
], ignore_index=True).sort_values('rmse_log_views').reset_index(drop=True)

fitted_pipelines['RandomForest_GridSearch'] = rf_best_pipe
results_df

```

```

In [ ]: # ---- Convert test predictions back to view scale for business interpretation
best_model_name = results_df.loc[0, 'model']
best_pipe = fitted_pipelines[best_model_name]
best_preds_log = best_pipe.predict(X_test)

compare = pd.DataFrame({
    'actual_views': np.exp1(y_test.values),
    'pred_views': np.exp1(best_preds_log)
})

compare['abs_error_views'] = (compare['actual_views'] - compare['pred_views']).abs

print('Best model:', best_model_name)
print(compare[['actual_views', 'pred_views', 'abs_error_views']].describe())

```

Notes for Final Report

- This notebook confirms whether your available variables support all four models.
- If a candidate feature is missing, the pipeline still runs with available features.
- Keep the leakage rule (no likes/comments when predicting views).
- Remember you can add hyperparameter tuning (GridSearchCV/RandomizedSearchCV) after baseline comparison.